

25 Python Programming Problems Key

Section 1: Core Python (Questions 1-8)

1. Variables and Data Types

```
# Solution 1: Variables and Data Types
name: str = "Satyasai Esarapu"
age: int = 25
height: float = 5.8
is_student: bool = True

print("Name:", name)
print("Age:", age)
print("Height:", height, "feet")
print("Is Student:", is_student)
```

2. Control Flow

```
# Solution 2: Control Flow with Type Hint
def check_number(num: int) -> str:
    if num > 0:
        return f"{num} is Positive"
    elif num < 0:
        return f"{num} is Negative"
    else:
        return f"{num} is Zero"

result: str = check_number(int(input("Enter a number: ")))
print(result)
```

3. For Loop - Multiplication Table

```
# Solution 3: Multiplication Table
def print_multiplication_table(num: int) -> None:
    for i in range(1, 11):
        print(f"{num} x {i} = {num * i}")

print_multiplication_table(5)
```

4. List Comprehension

```
# Solution 4: List Comprehension
squares: list[int] = [x**2 for x in range(1, 21)]
print("Squares from 1 to 20:", squares)
```

5. Dictionary Comprehension

```
# Solution 5: Dictionary Comprehension
cubes: dict[int, int] = {x: x**3 for x in range(1, 11)}
print("Cubes from 1 to 10:", cubes)
```

6. Function with Default Parameter

```
# Solution 6: Function with Default Parameter
def greet(name: str, greeting: str = "Hello") -> str:
    return f"{greeting}, {name}!"

print(greet("Satyasai"))
print(greet("Rahul", "Good Morning"))
```

7. Simple Decorator

```
# Solution 7: Simple Decorator
from typing import Callable

def simple_decorator(func: Callable[[], None]) -> Callable[[], None]:
    def wrapper() -> None:
        print("Function started")
        func()
        print("Function ended")
    return wrapper

@simple_decorator
def say_hello() -> None:
    print("Hello, welcome to Python learning!")

say_hello()
```

8. Function with Type Hints

```
# Solution 8: Function with Type Hints
from typing import List

def calculate_average(numbers: List[int]) -> float:
    if not numbers:
        return 0.0
    return sum(numbers) / len(numbers)

result: float = calculate_average([85, 90, 78, 92])
print("Average:", result)
```

Section 2: Data Structures, Error Handling & File Handling (Questions 9-17)

9. List Operations

```
# Solution 9: List Operations
fruits: list[str] = ["Apple", "Banana", "Mango", "Orange", "Grapes",
                    "Pineapple", "Kiwi", "Cherry"]

fruits.append("Watermelon")
fruits.append("Guava")
fruits.remove("Kiwi")

print("Final Fruit List:", fruits)
```

10. Tuple Unpacking

```
# Solution 10: Tuple and Unpacking
favorite_foods: tuple[str, ...] = ("Biryani", "Pizza", "Dosa", "Pasta", "Ice Cream")

food1, food2, food3, food4, food5 = favorite_foods

print("My favorite foods are:", food1, food2, food3, food4, food5)
```

11. Set Operations

```
# Solution 11: Set
numbers: list[int] = [1, 2, 2, 3, 4, 4, 5]
unique_numbers: set[int] = set(numbers)

print("Unique numbers:", unique_numbers)
print("Length of set:", len(unique_numbers))
```

12. Dictionary

```
# Solution 12: Dictionary
students: dict[str, int] = {
    "Satyasai": 92,
    "Rahul": 85,
    "Priya": 95,
    "Ankit": 78,
    "Neha": 88
}

highest_student: str = max(students, key=students.get)
print(f"Top Student: {highest_student} with {students[highest_student]} marks")
```

13. Error Handling

```
# Solution 13: Error Handling
def safe_divide(a: float, b: float) -> float | str:
    try:
        return a / b
    except ZeroDivisionError:
        return "Error: Cannot divide by zero!"
    except Exception as e:
        return f"Unexpected error: {e}"

print(safe_divide(10, 2))
print(safe_divide(10, 0))
```

14. JSON File Handling

```

# Solution 14: JSON File Handling with Type Hints
import json
from typing import Dict, Any

config_data: Dict[str, Any] = {
    "agent_name": "ResearchBot",
    "temperature": 0.7,
    "max_tokens": 2048
}

# Write file
with open("config.json", "w") as f:
    json.dump(config_data, f, indent=2)

# Read file safely
def load_config(file_path: str) -> Dict[str, Any] | None:
    try:
        with open(file_path, "r") as f:
            data: Dict[str, Any] = json.load(f)
            return data
    except (FileNotFoundError, json.JSONDecodeError) as e:
        print(f"Error loading config: {e}")
        return None

config: Dict[str, Any] | None = load_config("config.json")
print("Loaded Config:", config)

```

15. List Comprehension + Filtering

```

# Solution 15: List Comprehension
numbers: list[int] = [x for x in range(1, 51)]
even_numbers: list[int] = [x for x in numbers if x % 2 == 0]

print("First 10 even numbers:", even_numbers[:10])

```

**16. *args and kwargs

```

# Solution 16: *args and **kwargs
from typing import Any

def print_arguments(*args: Any, **kwargs: Any) -> None:
    print("Positional Arguments (*args):", args)
    print("Keyword Arguments (**kwargs):", kwargs)

print_arguments(10, 20, 30, name="Satyasai", city="Hyderabad", role="Developer")

```

17. AI Agent Class

```
# Solution 17: AI Agent Class with Type Hints
from typing import List

class AI Agent:
    def __init__(self, name: str, role: str) -> None:
        self.name: str = name
        self.role: str = role
        self.memory: List[str] = []

    def add_memory(self, thought: str) -> None:
        self.memory.append(thought)
        print(f"Memory added: {thought}")

    def show_memory(self) -> None:
        print(f"\nMemory of {self.name} ({self.role}):")
        for i, item in enumerate(self.memory, 1):
            print(f"{i}. {item}")

# Usage
agent: AI Agent = AI Agent("ResearchBot", "Information Gatherer")
agent.add_memory("Learned about Python basics")
agent.add_memory("Completed OOP concepts")
agent.show_memory()
```

Section 3: Object-Oriented Python (Questions 18-25)

18. Basic Class

```
# Solution 18: Person Class
class Person:
    def __init__(self, name: str, age: int) -> None:
        self.name: str = name
        self.age: int = age

    def introduce(self) -> None:
        print(f"Hi, I am {self.name} and I am {self.age} years old.")

p: Person = Person("Satyasai", 25)
p.introduce()
```

19. Class Method & Class Variable

```
# Solution 19: BankAccount
class BankAccount:
    total_accounts: int = 0

    def __init__(self, owner: str) -> None:
        self.owner: str = owner
        BankAccount.total_accounts += 1

    @classmethod
    def show_total_accounts(cls) -> None:
        print(f"Total Bank Accounts Created: {cls.total_accounts}")

acc1 = BankAccount("Satyasai")
acc2 = BankAccount("Rahul")
BankAccount.show_total_accounts()
```

20. Inheritance

```
# Solution 20: Inheritance
class Vehicle:
    def __init__(self, brand: str) -> None:
        self.brand: str = brand

    def start(self) -> None:
        print(f"{self.brand} vehicle is starting...")

class Car(Vehicle):
    def __init__(self, brand: str, model: str) -> None:
        super().__init__(brand)
        self.model: str = model

    def drive(self) -> None:
        print(f"Driving {self.brand} {self.model}")

my_car: Car = Car("Toyota", "Fortuner")
my_car.start()
my_car.drive()
```

21. Encapsulation

```
# Solution 21: Encapsulation
class Student:
    def __init__(self, name: str, email: str) -> None:
        self.name: str = name
        self.__email: str = email          # Private

    def get_email(self) -> str:
        return self.__email

s: Student = Student("Satyasai", "sai@example.com")
print("Name:", s.name)
print("Email:", s.get_email())
```

22. Polymorphism

```
# Solution 22: Polymorphism
class Dog:
    def speak(self) -> None:
        print("Woof Woof!")

class Cat:
    def speak(self) -> None:
        print("Meow Meow!")

animals: list = [Dog(), Cat()]

for animal in animals:
    animal.speak()
```

23. DataClass

```
# Solution 23: DataClass
from dataclasses import dataclass

@dataclass
class Book:
    title: str
    author: str
    year: int

book: Book = Book("Atomic Habits", "James Clear", 2018)
print(book)
```

24. Instance vs Class Method

```
# Solution 24: Calculator
class Calculator:
    version: str = "1.0"

    def add(self, a: int, b: int) -> int:
        return a + b

    @classmethod
    def show_version(cls) -> None:
        print(f"Calculator Version: {cls.version}")

calc: Calculator = Calculator()
print("Sum:", calc.add(15, 25))
Calculator.show_version()
```

25. Polymorphism with Shapes

```
# Solution 25: Polymorphism - Shapes
import math
from typing import List

class Shape:
    def area(self) -> float:
        raise NotImplementedError("Subclass must implement area()")

class Rectangle(Shape):
    def __init__(self, length: float, width: float) -> None:
        self.length: float = length
        self.width: float = width

    def area(self) -> float:
        return self.length * self.width

class Circle(Shape):
    def __init__(self, radius: float) -> None:
        self.radius: float = radius

    def area(self) -> float:
        return math.pi * self.radius ** 2

shapes: List[Shape] = [Rectangle(10, 5), Circle(7)]

for shape in shapes:
    print(f"Area = {shape.area():.2f}")
```

